RESEARCH RATIONALE / SURVIVAL PREORDERS

점수를 고르기 전에,
버려도 되는 정책부터 찾았다.

> "가장 안전한 정책"을 먼저 고르지 않았다.
> 어떤 안전 기준을 나중에 고르더라도 뒤집히지 않는 관계부터 물었다.

Survival curve 전체를 FSD preorder로 읽고, lossless shortlist, exact regret, no-fixed-dashboard, family-specific certificate까지 연결한 이론 연구의 발상과 증명 구조.

01 / ORDER

한 숫자가 아니라 전체 survival law가 강제하는 관계를 남긴다.

02 / DECISION

더 줄여야 한다면 그 대가를 worst-case regret로 정확히 가격한다.

03 / CERTIFICATE

모든 문제에 통하는 dashboard 대신, family가 요구하는 최소 표현을 만든다.

PROLOGUE / 개인적 출발점

나는 창조성의 본질이 무한원숭이정리에 있다고 생각했다.

> **인간은 시간적 제약 때문에 가능한 사고 공간을 편향시킬 뿐이다. 어찌 보면 그 편향 자체를 지능이라고 볼 수도 있다.**

내가 여기서 중요하게 본 건 **fluid intelligence**였다. 새로운 걸 만들고, 아무도 못 한 걸 생각하고, 기존에 없던 구조를 만드는 능력이다. 반대로 내가 **crystallized intelligence**라고 부르던 건 단순 암기 능력, 계산 능력, 정해진 규칙 안에서의 숙련에 가까웠다. 내 기준에서는 그건 진짜 지능이 아니라 가짜 지능이었다.

edge of chaos라는 관점도 같은 방향으로 읽었다. 완전히 닫힌 상태에서는 새로운 것이 잘 나오지 않고, 완전히 무작위인 상태에서는 축적이 안 된다. 중요한 건 임계점 가까이에서 기존 질서를 깨면서도 구조를 유지하는 탐색이라고 봤다.

그래서 발명과 발견도 구분했다.

논리적 사고, 연역적 사고, 기존 언어 안의 언어적 사고로 끝까지 밀어붙여 알아낼 수 있는 것은 발명이 아니라 발견이라고 봤다. 이미 닫혀 있는 **closed state** 안에서 아직 방문하지 않은 지점을 찾는 일이다.

내가 생각한 발명은 **새로운 언어를 만드는 일**이었다. 기존 표현으로는 보이지 않던 문제를 보이게 하고, 기존 좌표계에는 없던 방향을 열어야 한다고 생각했다.

당시의 질문

무한원숭이정리식의 충분한 탐색이라면 상대성이론을 만들거나 리만가설의 증명을 생성할 수 있어도, 기존 언어와 확률분포 안에서 autoregressive하게 움직이는 LLM은 그런 종류의 발명에 구조적인 한계가 있지 않을까?

ORIGIN / 어려운 수학 문제에서 본 이상한 현상

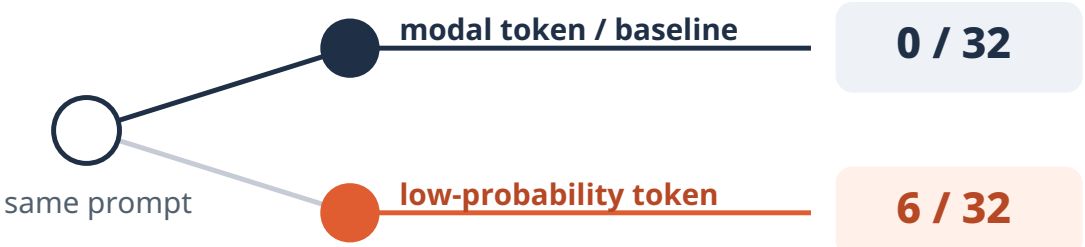
가장 그럴듯한 경로와 정답으로 가는 경로가 다를 수 있었다.

메모리를 끄고 같은 수학 난제를 같은 prompt로 반복했다. 쉬운 문제는 비슷한 풀이로 모였지만, 문제가 어려워질수록 출력이 여러 방향으로 흩어지는 느낌이 강해졌다.

> **그래서 난제일수록 정답률이 probability의 mode에서만 높지 않고, 낮은 확률 영역에서 다시 살아나는 U-shape가 있을 거라고 직관적으로 추측했다.**

문제는 이 가설 자체를 직접 실험할 방법이 없다는 것이었다. "난이도"도 애매하고, "가능한 모든 접근법의 분포"는 더더욱 정의할 수 없다. 그래서 전체 분포를 증명하려 하지 않고 **분포를 인위적으로 뒤집어보기**로 했다.

가장 작은 개입: 첫 번째 token



약 200개 경시대회 문제 × 조건별 32회. 전체 평균에서는 baseline이 가장 높았지만, 일부 문제에서는 baseline이 0/32인 동안 낮은 확률 branch에서 반복적으로 정답이 나왔다. 이 관찰은 paper의 theorem이 아니라, 문제를 다시 정의하게 만든 선행 탐색이었다.

중요한 건 "tail이 더 좋다"가 아니었다. 평균적으로는 mode가 더 좋았다. 그런데 몇몇 어려운 문제에서는 정답을 포함한 probability mass가 mode 밖에 실제로 존재했다.

FAILURE / 처음 세운 분석법의 모순

“접근법”도, “결정적 token”도 자연스러운 좌표가 아니었다.

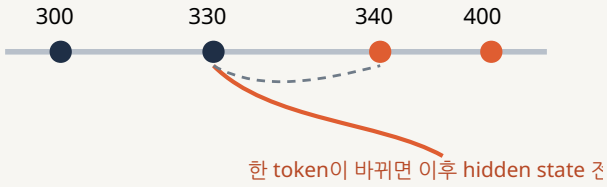
1. 접근법을 어떻게 정의할 건가?

Lean 표현으로 나눌지, 문장으로 나눌지, 인간이나 AI가 “사실상 같은 접근”이라고 판정하게 할지 정할 수 없었다. 어떤 선택을 해도 먼저 내가 representation을 만든다.

첫 번째 모순
접근법을 측정하려면 먼저 접근법을 정의해야 하고, 그 정의가 분석 결과를 바꾼다.

2. token 구간을 좁히면 insight를 찾을 수 있나?

처음에는 300~400 token 사이에서 continuation 정답률이 뛰면 300, 310, 320…으로 좁히고, 330~340에서 다시 뛰면 330, 331…로 좁힐 생각이었다.



> 내가 원한 counterfactual은 “나머지 reasoning state는 같은데 이 생각 하나만 없었다면?”이었다. autoregressive model에서는 그 상태를 자연스럽게 만들 수 없었다.

330에서 340으로 갈 때 정답률이 뛰어도 337번째 token이 원인이라는 뜻이 아니다. 앞선 수백 token에서 누적된 정보가 그 지점에서 충분해졌을 수도 있고, 이미 다른 branch에 들어간 결과일 수도 있다.

여기서 문제가 일반화됐다.

대상보다 먼저 좌표를 정하면, 나중에 보이는 구조가 대상의 구조인지 좌표가 만든 구조인지 구분하기 어렵다.

CONCEPTUAL TOOLS / 이미 알고 있던 두 개의 언어

이때 두 책의 개념이 문제를 훨씬 선명하게 만들었다.

직접 theorem을 가져온 게 아니라, 무엇을 먼저 물어야 하는지 정리해준 배경이었다.

The Art of Quantitative Finance

하나의 분포를 평균, tail risk, 특정 threshold 같은 서로 다른 요약으로 읽으면 같은 대상의 순위가 바뀔 수 있다는 사고에 익숙했다.

> **분포가 있는데 왜 먼저 한 숫자로 줄여야 하지?**

이 논문에서는 이 질문이 catastrophe-time distribution과 scalar safety metric의 관계로 옮겨갔다.

Theory of Games and Economic Behavior

utility, domination, max-min/minimax라는 언어가 있었다. 모든 허용된 평가에서 계속 밀리는 선택지는 먼저 버리고, 남은 선택의 최악의 손실을 게임으로 볼 수 있다.

> **winner를 고르기 전에 dominated option부터 지울 수 있지 않나?**

이 직관이 FSD maximal classes와 worst-case shortlist regret를 생각하는 데 직접적으로 맞았다.

두 개를 합치면 질문은 하나가 된다.

"어떤 safety score가 맞는가?"가 아니라, "score를 고르기 전에도 뒤집히지 않는 관계가 무엇인가?"

PROBLEM / SCALARIZATION

Safety에서도 똑같은 문제가 보였다. 분포 하나, 점수 세 개, winner 세 명.

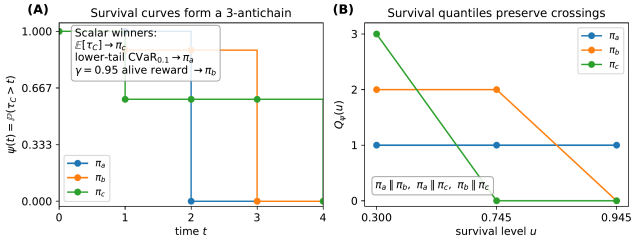
정책 하나에는 catastrophe time의 전체 distribution이 있다. 그런데 보통은 평균 생존시간, lower-tail CVaR, discounted survival, 특정 deadline 생존확률 같은 scalar로 먼저 압축한다.

survival curve가 교차하면 이 압축 방식이 순위를 만든다. 즉 **"가장 안전한 정책"이라는 답 안에 이미 metric 선택이 들어가 있다.**

> **그래서 metric을 하나 더 제안하는 대신, metric 선택 이전의 order를 보려고 했다.**

$$\psi_i(t) = P(\tau_i > t)$$

정책 i가 deadline t까지 catastrophe 없이 살아 있을 확률



논문의 실제 figure. 같은 세 catastrophe-time law에서 mean, lower-tail CVaR, discounted survival이 서로 다른 unique winner를 고른다.

CORE PROOF 01 / PROPOSITION 3

“누가 최고인가?”보다 먼저, “누구를 확실히 버려도 되는가?”를 물었다.

FSD는 아주 단순하다.

A가 1단계에서도 B보다 안전하고, 2단계에서도 안전하고, 모든 deadline에서 더 높은 survival probability를 가지면 A가 B를 FSD-dominate한다.

$$i \succcurlyeq_{s} j \iff \psi_i(t) \geq \psi_j(t) \quad \text{for every } t$$

곡선이 교차하면 억지로 순위를 만들지 않는다. 그냥 incomparable로 남긴다. 중요한 건 그 상태를 실패로 보지 않는 것이다.

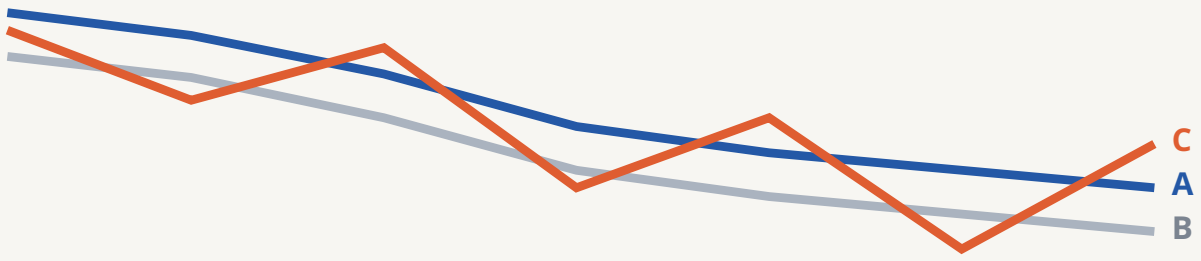
> **정보가 partial order까지만 말해주면, partial order까지만 말한다.**

왜 maximal set이 lossless인가?

1. finite partial order에서 non-maximal policy 위에는 maximal dominator가 하나 이상 있다.
2. objective가 FSD-monotone이면 위로 올라갈 때 objective가 나빠질 수 없다.
3. 따라서 non-maximal policy를 maximal dominator로 바꿔도 optimum을 잃지 않는다.

결론

preference를 아직 정하지 않았는데도 확실히 제거할 수 있는 정책만 먼저 제거할 수 있다.



A는 B를 완전히 덮는다 → B는 버려도 됨. C는 교차한다 → 함부로 버리면 안 됨.

CORE PROOF 02 / THEOREM 5

더 줄여야 한다면,
그 대가를 "최악의 preference"로 정확히 가격했다.

maximal policy가 여러 개라면 운영자는 결국 몇 개만 남기고 싶을 수 있다. 이때 임의의 metric을 다시 고르는 대신, **아직 모르는 monotone utility가 나를 가장 불리하게 공격한다고 가정**했다.

> 나: shortlist를 먼저 고른다. / 상대: 그 shortlist가 가장 후회스럽게 보이는 preference를 고른다.

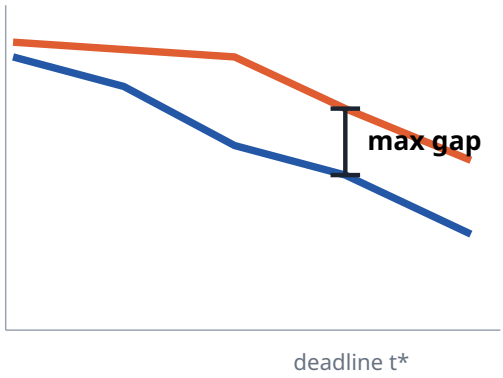
증명에서 먼저 한 일

복잡한 utility 함수 전체를 상대하지 않고, deadline마다 주는 weight로 다시 썼다.

$$E[u(Y)] = u(0) + \Sigma_t [u(t+1)-u(t)]\ \psi(t)$$

u가 nondecreasing이므로 weight는 모두 nonnegative. normalize하면 합이 1이어서 simplex가 된다.

그러면 worst-case utility는 어렵지 않다. 내가 고른 정책이 다른 정책보다 가장 크게 밀리는 deadline에 weight를 몰아주면 된다.



정책 하나의 worst-case regret = 다른 survival curve가 위로 가장 많이 튀어나온 높이.

핵심 reduction "무한한 preference space"를 deadline-weight simplex로 바꾸자, worst case는 extreme point 하나로 줄었다. shortlist가 여러 개면 같은 구조가 finite primal-dual LP가 된다.

CORE PROOF 03 / THEOREM 9 + COROLLARY 10

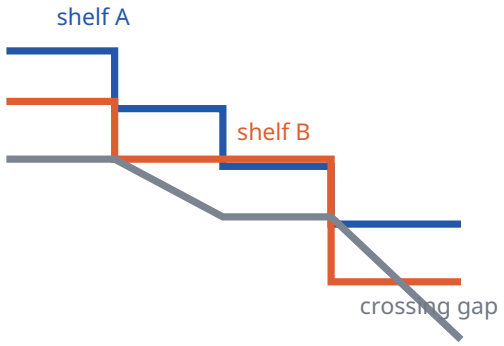
“한 점수가 부족하면 10개 쓰면 되잖아?” 그 반론 자체를 막고 싶었다.

특정 dashboard가 실패하는 예제를 하나 만드는 건 약하다. 더 좋은 scalar들을 고르면 된다는 반론이 남는다. 그래서 질문을 universal하게 바꿨다.

> 어떤 finite partial order라도 survival curves 안에 심을 수 있다면, 미리 정한 finite-dimensional dashboard는 언젠가 반드시 부족해진다.

증명 아이디어: “shelf”로 poset을 코딩한다

1. poset의 각 원소마다 survival curve에 작은 shelf 하나를 배정한다.
2. e가 g보다 위에 있을 때만 e의 해당 shelf를 올린다.
3. comparability는 curve containment로, incomparability는 반대 방향의 gap으로 남는다.
4. 따라서 임의의 finite poset을 survival quotient로 realization할 수 있다.



order dimension은 family에 따라 얼마든지 커질 수 있다. 그러므로 고정된 d개의 scalar로 모든 family를 coordinatewise 표현하는 universal dashboard는 존재하지 않는다.

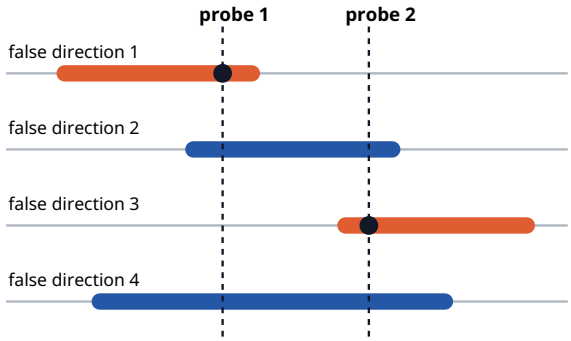
이 theorem의 value는 “좋은 score를 찾았다”가 아니라, score를 미리 고르는 전략 자체에 한계가 있다는 걸 증명한 것이다.

CORE PROOF 04 / THEOREM 12

Universal dashboard가 없다면, 이번 family에 필요한 최소 표현을 만들면 된다.

i가 j를 dominate한다는 주장이 거짓이라면, survival level 어딘가에서는 반드시 quantile order가 뒤집힌다. 그 구간이 **witness set**이다.

> **거짓 비교마다 "현행범으로 잡히는 구간"이 있다. 모든 거짓말을 최소 한 번씩 잡는 관측점만 고르면 된다.**



모든 witness interval을 적어도 한 번 찌르는 최소 point set = hitting set.

NP-hardness가 뜻하는 것

어려운 건 **audit 자체가 아니라 가장 작은 certificate를 찾는 것**이다.

preorder, quotient, maximal classes는 exact curves가 있으면 직접 계산할 수 있다. 최소 point grid는 NP-complete지만, greedy Set Cover는 order를 정확히 보존한 채 certificate 크기만 근사한다.

비유
도시의 모든 사건을 확인하는 건 가능하다. "모든 사건을 잡는 최소 CCTV 수"를 찾는 게 어렵다.

CORE PROOF 05 / COROLLARY 7

정확한 순위를 아는 것과 좋은 결정을 하는 것은 다른 문제다.

두 곡선이 0.000001 차이라면?

"누가 위인가?"를 정확히 판정하려면 그 작은 gap을 끝까지 식별해야 한다. exact preorder recovery가 margin-dependent인 이유다.

하지만 실제 목적이 "좋은 k개 shortlist를 남겨라"라면 그 미세한 순서를 다 알 필요가 없다. 필요한 건 **그 불확실성이 decision regret를 얼마나 움직이는지**다.

> **world의 order를 완전히 복원하는 것보다, 잘못 고를 때 실제로 얼마를 잃는지가 더 직접적인 decision target일 수 있다.**

왜 +4e가 나오는가?

1. 각 survival curve를 sup-norm으로 최대 e만큼 틀리게 추정했다고 하자.
2. 두 curve의 차이는 양쪽이 각각 e씩 움직일 수 있으니 shortlist regret는 최대 2e 흔들린다.
3. empirical optimum과 true optimum을 비교할 때 이 2e를 두 번 건너므로 총 4e다.

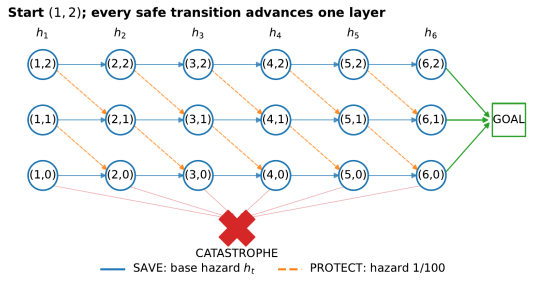
$$R(\hat{S}_k) \le R_k^* + 4e$$

이게 이 연구에서 가장 중요한 decision-level 결과다.

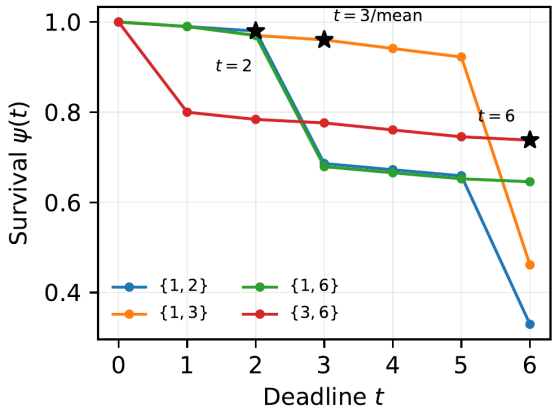
exact identification은 작은 margin에 민감하지만, shortlist decision은 positive order margin 없이도 안정적으로 보장할 수 있다.

EXACT BENCHMARK / 22 POLICIES

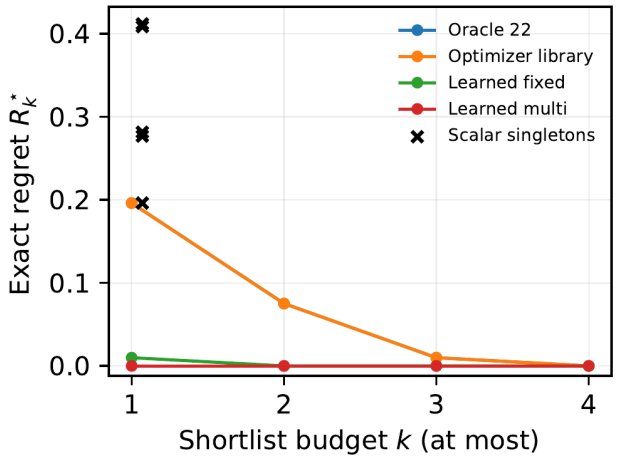
이론이 실제로 무엇을 바꾸는지 완전 열거 가능한 문제에서 확인했다.



Proper layered shield-scheduling MDP. 모든 deterministic schedule을 exact rational arithmetic으로 열거할 수 있다.



4개의 maximal classes가 pairwise incomparable하다. 서로 다른 표준 criterion이 서로 다른 maxima를 선택한다.



Shortlist budget k에 따른 exact worst-case regret.

22 deterministic schedules

4 pairwise-incomparable maximal classes

0.1961

최적의 single-policy shortlist도 worst-case normalized utility range의 19.61%를 잃을 수 있다.

0 regret at k = 4

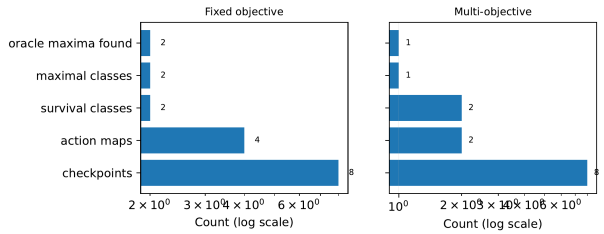
네 maximal class를 모두 남기면 어떤 monotone safety preference를 나중에 고르더라도 손실이 없다.

OPERATIONAL SCOPE / FINITE FAMILY

finite-family는 단순한 제한이 아니라 실제 audit의 단위였다.

배포 직전에는 "모든 가능한 policy" 중 하나를 고르지 않는다. 실제로 테이블 위에 올라오는 것은 checkpoint archive, seed sweep, hyperparameter sweep, controller library처럼 이미 만들어진 유한한 후보들이다.

> 이론의 object를 전체 policy space가 아니라, 실제 deployment table로 잡았다.



학습된 checkpoint corpus를 action maps와 survival-equivalence classes로 압축하는 실제 paper figure.

960 checkpoints

175 distinct action maps

10 distinct survival laws

archive-wide audit은 exact oracle family의 네 FSD-maximal class를 모두 남겼지만, final-checkpoint-only selection은 하나를 놓쳤다.

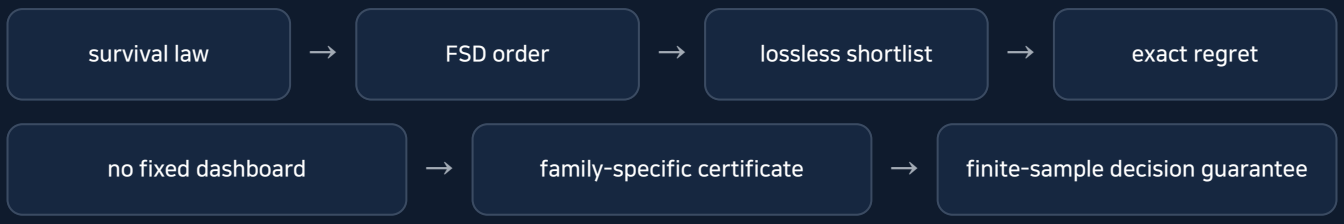
즉 audit은 "학습을 대신하는 objective"가 아니라, 학습이 만든 후보 집합을 배포 전에 다시 읽는 층이다.

checkpoint archive → survival laws → FSD quotient → lossless shortlist

WHAT THE PAPER IS ABOUT

정책의 완전한 순위를 알아내는 논문이 아니다.

> preference가 정해지기 전에 무엇을 안전하게 버릴 수 있고,
> 더 버리면 얼마를 잃는지를 계산하는 audit theory다.



핵심 증명에서 내가 잡은 reduction

maximality → finite partial order

worst-case utility → deadline-weight simplex

universal impossibility → arbitrary poset realization

exact certificate → witness-set hitting set

decision stability → 2e + 2e = 4e

Contribution boundary

FSD, DKW, order dimension, Set Cover는 classical하다.

새로 만든 건 이 도구들의 이름이 아니라, finite policy library를 "lossless reduction → priced compression → certifiable recovery"로 연결한 family-level 구조다.

Figures and empirical values are from the accompanying paper repository and revised manuscript.